Case Study – Theory of Computation

Student Details

Name: V Venkat Akhilesh

Roll Number: 2420030705

Title of the Case Study

Mobile Number Validation using Regular Expressions and DFA

Introduction / Overview

Mobile number validation is a common real-world requirement in web forms, banking applications, and OTP-based authentication systems, where an input string must be checked before it is accepted as a valid 10-digit Indian mobile number. In practice, systems need to instantly reject malformed entries such as numbers that are too short, too long, contain letters, or start with an invalid digit. This case study models that validation problem using Regular Expressions and an equivalent Deterministic Finite Automaton (DFA), showing how a formal language-theory tool can enforce a real input-format rule. The alphabet considered is the set of digits {0,1,...,9}, and the language accepted is exactly the set of strings of length 10 whose first digit lies between 6 and 9, matching standard Indian mobile number conventions.

Problem Statement and Objectives

Problem Statement: Given an input string, determine whether it represents a valid 10-digit mobile number that starts with a digit from 6 to 9, using a Regular Expression and a corresponding DFA.

- To design a Regular Expression that precisely defines the language of valid mobile numbers.
- To construct an equivalent DFA that accepts exactly the strings matching this Regular Expression.
- To build the state transition table for the DFA and trace its behaviour on sample inputs.
- To verify the design by testing valid and invalid mobile number strings against the DFA.

Case Study Description

A mobile number entered into a form is treated as an input string over the alphabet {0-9}. For the number to be valid, two conditions must hold simultaneously: the string must be exactly 10 characters long, and the very first character must be one of 6, 7, 8, or 9 (matching the numbering scheme used by Indian mobile operators). Any other digit as the first character, or any deviation in length, must be rejected. This is naturally expressed as a Regular Expression and implemented as a DFA, where each state tracks how many valid digits have been consumed so far, allowing the automaton to accept or reject the string as soon as the last character is processed.

Automata / RE Design

Regular Expression:

```
[6-9][0-9]{9}
```

This expression means: the first symbol must be a digit from 6 to 9, followed by exactly nine more symbols, each any digit from 0 to 9 — giving a total length of exactly 10.

DFA Design (informal state diagram):

q0 --[6-9]--> q1 --[0-9]--> q2 --[0-9]--> q3 --[0-9]--> q4 --[0-9]--> q5 --[0-9]--> q6 --[0-9]--> q7 --[0-9]--> q8 --[0-9]--> q9 --[0-9]--> q10(final). Any invalid transition (wrong first digit, or any digit after q10) leads to the trap state qt, which loops on itself for every remaining symbol.

Q = {q0, q1, q2, ..., q10, qt}, Σ = {0,1,...,9}, Start state = q0, Final state = {q10}, qt is the non-accepting dead/trap state.

State Transition Table:

State	Digits 0-5	Digits 6-9	Digits 0-9 (later positions)	Type	Notes
q0 (start)	qt	q1	-	Start	1st digit must be 6-9
q1	-	-	q2	Intermediate	1 digit consumed
q2	-	-	q3	Intermediate	2 digits consumed
q3	-	-	q4	Intermediate	3 digits consumed
q4	-	-	q5	Intermediate	4 digits consumed
q5	-	-	q6	Intermediate	5 digits consumed
q6	-	-	q7	Intermediate	6 digits consumed
q7	-	-	q8	Intermediate	7 digits consumed
q8	-	-	q9	Intermediate	8 digits consumed
q9	-	-	q10	Intermediate	9 digits consumed
q10 (final)	-	-	qt	Accepting	10th digit consumed; any extra digit -> trap
qt (trap)	qt	qt	qt	Dead/Reject	Absorbing state for invalid input

Results and Outcomes

The DFA was traced on several sample strings to confirm it accepts exactly the intended language:

Input String	DFA Result	Reason
9876543210	Accepted	Starts with 9, exactly 10 digits

Input String	DFA Result	Reason
7894561230	Accepted	Starts with 7, exactly 10 digits
1234567890	Rejected	Starts with 1 (not in 6-9), trapped at q0
98765432	Rejected	Only 8 digits; ends in intermediate state, not final
98765432101	Rejected	11 digits; 11th digit forces transition out of q10 into trap
+919876543210	Rejected	Contains non-digit / country-code characters not in alphabet {0-9}

Outcome: The DFA correctly accepts all and only those strings that are exactly 10 digits long and begin with 6, 7, 8, or 9, exactly matching the Regular Expression [6-9][0-9]{9}. This confirms that the Regular Expression and the constructed DFA are equivalent, and demonstrates how a real-world input-validation rule (mobile number formatting) can be modelled precisely using Theory of Computation concepts. It was also observed that even a single misplaced or extra character forces the automaton into the trap state, which is exactly the behaviour required of a robust input validator.